Diplomatura en programación web full stack con React JS

Módulo 6:

Administración, API Rest y despliegue

Unidad 3:

CRUD (parte 3)



Presentación

En esta unidad implementaremos la funcionalidad de subir archivos a nuestro servidor. Junto a esto utilizaremos el servicio de Cloudinary para almacenar y transformar las imágenes subidas. Crearemos además una API Rest para que nuestro frontend consuma los datos cargados en el administrador.



Objetivos

Que los participantes logren…

- Implementar las funcionalidades necesarias para realizar la subida de archivos al servidor.
- Conectar el sitio con el servicio de Cloudinary para almacenar las imágenes subidas y obtener versiones recortadas de las mismas.
- Crear una API Rest en Express y consumirla desde el frontend.



Bloques temáticos

1. Subida de imágenes.
2. Creación de API Rest.
3. Consumo de API desde el frontend.

1. Subida de imágenes

Paso 1

Nuestro sitio web permitirá al administrador subir imágenes a las novedades que ya tenemos implementadas. Para esto será necesario hacer algunas modificaciones en el código y estructura de base de datos que ya tenemos.

Necesitaremos primero instalar algunas dependencias nuevas:

express-fileupload

La dependencia express-fileupload nos va a permitir, mediante un middleware, capturar y manipular archivos subidos mediante formularios a nuestro sitio web.

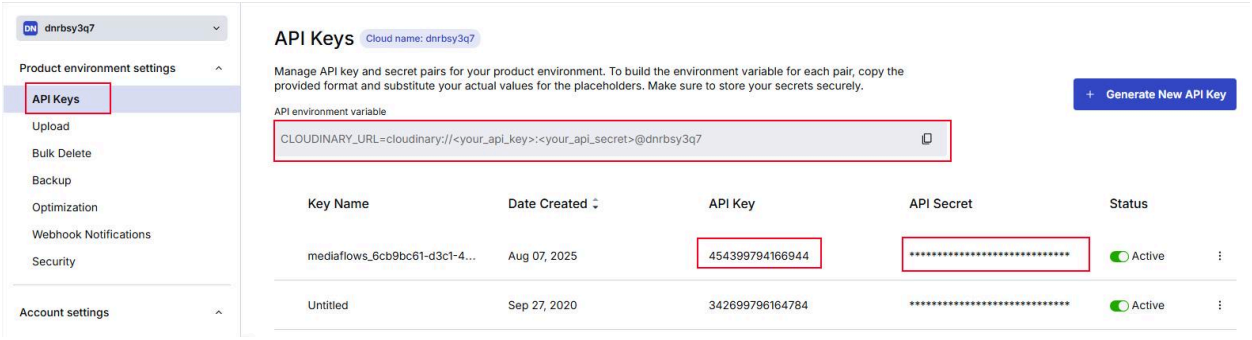
```
npm i express-fileupload
```

cloudinary

Utilizaremos el servicio gratuito de Cloudinary (https://cloudinary.com/) para alojar y manipular las imágenes que se vayan subiendo a las novedades. La dependencia la instalamos de la siguiente manera:

```
npm i cloudinary
```

Para poder utilizar el servicios necesitaremos registrarnos en el sitio y, una vez terminado este proceso, copiar el valor de API Environment variable a nuestro archivos .env con el nombre **CLOUDINARY_URL**




```
.env
1  MYSQL_HOST=localhost
2  MYSQL_DB_NAME=cerveceria
3  MYSQL_USER=root
4  MYSQL_PASSWORD=
5
6  CLOUDINARY_URL=cloudinary://342699796164784:O06-Eck-Cr
7
```

Agregar imágenes a la novedades

Paso 1

Incluimos la dependencia de **express-fileupload** y los configuramos como middleware de nuestra aplicación de express. Es importante configurar el uso de archivos temporales a fin de que podamos, una vez subidos, enviarlos al servidor de Cloudinary donde quedarán almacenados.

```
var fileUpload = require('express-fileupload');


app.use(fileUpload({
  useTempFiles: true,
  tempFileDir: '/tmp/'
}));
```

Paso 2

Agregamos en la tabla de novedades, la columna **img_id**, con el tipo de datos varchar, cantidad de caracteres 250 y tildamos la opción NULL (para que permitir la carga de novedades sin imágenes)

Paso 3

En el archivo **view/admin/agregar.hbs**, agregamos en la etiqueta **<form>**: el atributo enctype con el valor multipart/form-data. Esto hará que el formulario envíe no solo los campos que veníamos usando hasta ahora, sino que también enviará los archivos que incluyamos.

```
<form action="/admin/novedades/agregar" method="post"
enctype="multipart/form-data" />
```

Debajo de la etiqueta input cuerpo agregamos el **input** del tipo **file** para poder adjuntar la imagen y le ponemos de nombre imagen.

```
<div class="form-group">
   <label>Imagen: <input type="file" name="imagen"
                  id="imagen" /></label>
</div>
```

Paso 4

Modificamos nuestro archivo **routes/admin/novedades.js** e incluimos tanto la dependencia de cloudinary como la de util que usaremos para convertir el método upload en una función asincrónica.

```
var util = require('util');
var cloudinary = require('cloudinary').v2;

const uploader = util.promisify(cloudinary.uploader.upload);
```

Dentro del controlador router.post (**/agregar**) vamos a verificar, antes de enviar la imagen a nuestro modelo, inicializamos una variable **img_id** como vacía además chequeamos si existe el objeto files dentro de la petición y si incluye uno o más objetos dentro suyo.

Esta es la función que realiza express-fileupload; agregar en la petición el objeto files, y dentro de él facilitarnos todos los inputs del tipo file que se hayan enviado con su nombre.

```
router.post('/agregar', async (req, res, next) => {
    try {
        var img_id = '';
        if (req.files && Object.keys(req.files).length > 0) {
            imagen = req.files.imagen;
            img_id = (await uploader(imagen.tempFilePath)).public_id;
        }

        if (req.body.titulo != "" && req.body.subtitulo != ""
            && req.body.cuerpo != "") {

            await novedadesModel.insertNovedad({
                ...req.body,
                img_id
            });

            res.redirect('/admin/novedades')
        }
```

En caso de que se hayan enviado archivos, buscaremos uno llamado imagen (recordar el atributo name del input que agregamos) y utilizaremos la ruta temporal donde fue almacenado en nuestra computadora como archivo de origen para enviar al uploader de Cloudinary. Por último, al completarse la subida a cloudinary guardamos el valor de la propiedad public_id en nuestra variable img_id.

Este proceso hace que la variable img_id, que enviamos junto con el resto de los campos a la base de datos, solo contenga un valor (el id publico que cloudinary asignó a nuestra imagen y que necesitaremos más adelante para obtenerla y manipularla) en caso de que se haya subido una imagen. En caso contrario, enviaremos un valor vacío y la novedad quedará almacenada sin imagen.

Mostrar imágenes en el listado del administrador

Paso 1

En el archivo **routes/admin/novedades.js** agregamos la lógica necesaria para enviar junto con cada novedad la imagen que se haya subido en caso de haber una o nada en caso de no haberse subido una imagen a esa novedad.

Para esto utilizaremos el método image de cloudinary que nos devuelve una etiqueta img html completa.

```
router.get('/', async function (req, res, next) {
    var novedades = await novedadesModel.getNovedades();

    novedades = novedades.map(novedad => {
        if (novedad.img_id) {
            const imagen = cloudinary.image(novedad.img_id, {
                width: 100,
                height: 100,
                crop: 'fill'
            });
            return {
                ...novedad,
                imagen
            }
        } else {
            return {
                ...novedad,
                imagen: ''
            }
        }
    });

    res.render('admin/novedades', {
        layout: 'admin/layout',
        usuario: req.session.nombre,
        novedades
    });
});
```

Paso 2

Finalmente en el archivo **view/admin/novedades.hbs** agregaremos en el bucle each la variable de la imagen con triple llave debido a que puede contener HTML (la etiqueta img).

```
<tr>
    <th scope="col">#</th>
    <th scope="col">Título</th>
    <th scope="col">Imagen</th>
    <th class="text-center" scope="col">Acciones</th>
</tr>

<tr>
    <th scope="row">{{id}}</th>
    <td>{{ titulo }}</td>
    <td>{{{imagen}}}</td>
</tr>
```

Modificar una novedad con imagen

Paso 1

Modificamos **views/admin/modificar.hbs** e incluimos el input para poder enviar la imagen. A diferencia del formulario de agregar, ahora incluiremos también un input del tipo checkbox para permitir a los usuarios eliminar la imagen asociada a la novedad.

Es necesario también que incluyamos un input del tipo hidden para enviar junto con el formulario, el id de la imagen actual de la novedad en caso de haber una.

```
<div class="form-group">
    <p>
        <label>Imagen: <input type="file" name="imagen"
                id="imagen" /></label>
    </p>
    <p>
        <label><input type="checkbox" name="img_delete"
                id="img_delete" value="1">
                Eliminar imagen actual?</label>
        <input type="hidden" name="img_original"
                value="{{novedad.img_id}}">
    </p>
</div>
```

Paso 2

En el archivo **routes/admin/novedades.js** agregamos el método **destroy** de cloudinary que nos permite eliminar imágenes del servicio. Al igual que hicimos con el uploader, utilizaremos la librería util para convertirlo en un método asíncrono.

El código para manejar la modificación de la imagen en la novedad puede parecer complejo a primera vista pero no lo es. Primero inicializamos la variable img_id con el valor que haya venido del input hidden y la variable borrar_img_vieja que usaremos para saber si tenemos que llamar al método

destroy. De esta forma, pase lo que pase, al enviar el valor a la base de datos, preservaremos el valor original.

A continuación revisamos si el usuario decidió eliminar la imagen novedad, en cuyo caso asignamos null a img_id para limpiar el valor de la base de datos.

Si el usuario **no tildó la casilla** para eliminar la imagen actual repetimos el proceso que hicimos en agregar para ver si se envió una imagen junto con el formulario. En caso afirmativo la subimos a cloudinary, almacenamos el id publico en nuestra variable img_id y seteamos borrar_img_vieja a verdadero.

En el último paso revisamos el valor de borrar_img_vieja y en caso de ser verdadero, ya sea porque **el usuario tilo la casilla o subió una imagen nueva**, llamamos al método destroy para eliminar la imagen antigua de cloudinary.

Al finalizar todo este proceso, el valor de img_id contendrá una de tres opciones según el camino que haya seguido el código, null, el valor del input hidden o el id publico de una nueva imagen.

```
const destroy = util.promisify(cloudinary.uploader.destroy);


router.post('/modificar', async (req, res, next) => {
    try{
        let img_id = req.body.img_original;
        let borrar_img_vieja = false;
        if (req.body.img_delete === "1") {
            img_id = null;
            borrar_img_vieja = true;
        } else {
            if (req.files && Object.keys(req.files).length > 0) {
              imagen = req.files.imagen;
              img_id = (await
              uploader(imagen.tempFilePath)).public_id;
              borrar_img_vieja = true;
            }
        }
        if (borrar_img_vieja && req.body.img_original) {
            await (destroy(req.body.img_original));
        }
```

Paso 3

En la variable **obj** agregamos **img_id** para enviarlo a la base de datos junto con el resto de los campos.

```
var obj = {
    titulo:req.body.titulo,
    subtitulo:req.body.subtitulo,
    cuerpo: req.body.cuerpo,
    img_id
}
```

Para eliminar una imagen

Paso 1

En el archivo **routes/admin/novedades.js** dentro del controlador de la ruta que elimina las novedades vamos a verificar primero si la novedad tiene una imagen, en cuyo caso, como vamos a eliminar la novedad de nuestra base de datos, la eliminamos tambien de cloudinary con el método **destroy.**

```
router.get('/eliminar/:id', async (req, res, next) => {
    var id = req.params.id;

    let novedad = await novedadesModel.getNovedadById(id);
    if (novedad.img_id) {
        await (destroy(novedad.img_id));
    }

    await novedadesModel.deleteNovedadesByID(id);
    res.redirect('/admin/novedades')
});
```

2. Creación de API Rest

Las API Rest son interfaces que las aplicaciones exponen para permitir la comunicación con otras y/o el consumo de datos controlado por parte de terceros. Muchas aplicaciones exponen sus servicios mediante la implementación de APIs Rest, permitiéndonos utilizar funcionalidades o acceder a datos a los cuales de otra forma sería imposible acceder.

En el caso de nuestra aplicación crearemos un endpoint (así se denomina cada URL que compone una API Rest) para exponer las novedades que se hayan cargado en la base de datos mediante el administrador. Esta sencilla API permitirá que nuestro sitio o cualquier otra aplicación (podría ser otro sitio o una aplicación de escritorio o para celulares) consuma nuestros datos

CORS

CORS (del inglés Cross-Origin Resource Sharing) es un mecanismo que utiliza cabeceras HTTP adicionales para permitir que un sitio obtenga permiso para acceder a recursos seleccionados desde un servidor, en un origen distinto (dominio) al que pertenece. Es una práctica de seguridad común para prevenir el uso no autorizado de recursos.

Instalamos la dependencia necesaria con el siguiente comando.

```
npm i cors
```

Luego importamos la nueva dependencia en nuestro archivo **app.js.**

```
var cors = require('cors');
```

Para una mejor organización, vamos a separar todos los controladores para las rutas de nuestra API en un nuevo ruteador de express. Para eso creamos un archivo nuevo **routes/api.js** y lo importamos también en nuestro archivo **app.js.**

```
var apiRouter = require('./routes/api');

app.use('/api', cors(), apiRouter);
```

Como se puede ver nuestro ruteador se encargará de todas las rutas que empiecen con **/api** e implementará a modo de middleware la librería de CORS que instalamos. Esto permitirá que cualquier ruta de nuestra API pueda ser consumida desde dominios diferentes al que aloje a nuestra aplicación.

El contenido de **routes/api.js** es el siguiente.

```
var express = require('express');
var router = express.Router();
var novedadesModel = require('./../models/novedadesModel');

router.get('/novedades', async function (req, res, next) {
    let novedades = await novedadesModel.getNovedades();

    res.json(novedades);
});

module.exports = router;
```

Devolver los datos de nuestra base de datos mediante la API, como se puede apreciar, es muy sencillo. Sólo debemos devolver como respuesta JSON el listado de novedades que nuestro modelo ha generado.

Para poder probar nuestra API podemos acceder a la ruta **http://localhost:3000/api/novedades** utilizando Chrome u otro navegador. Dependiendo de la configuración o extensiones que tengamos instaladas podemos ver el contenido con un formato legible o el texto todo junto.

Una alternativa muy utilizada cuando desarrollamos APIs con muchos endpoints y necesitamos testearlas, más allá de que tengamos una aplicación que las consuma, es una aplicación llamada Postman. Ésta se puede descargar de https://www.postman.com/downloads/ para Windows, macOS o Linux. Allí también se encuentra la documentación con explicaciones y ejemplos de su uso.

Manipulación de imágenes

Si prestamos atención hasta ahora solo estamos devolviendo en nuestra API el ID de la imagen que nos devolvió Cloudinary. Si queremos devolver la url de dicha imagen solamente debemos incorporar la librería de Cloudinary a nuestro código y consultar las urls de las imágenes de las cuales tenemos el ID.

Cabe destacar que Cloudinary además soporta varios métodos de manipulación de imágenes como redimensión, recorte o efectos que se pueden aplicar de forma muy sencilla en nuestro código.

```
var express = require('express');
var router = express.Router();
var novedadesModel = require('./../models/novedadesModel');
var cloudinary = require('cloudinary').v2;


router.get('/novedades', async function (req, res, next) {
    let novedades = await novedadesModel.getNovedades();

    novedades = novedades.map(novedades => {
        if (novedades.img_id) {
            const imagen = cloudinary.url(novedades.img_id, {
                width: 960,
                height: 200,
                crop: 'fill'
            });
            return {
                ...novedades,
                imagen
            }
        } else {
            return {
                ...novedades,
                imagen: ''
            }
        }
    });

    res.json(novedades);
});


module.exports = router;
```

En nuestro ejemplo estamos aplicando un recorte (crop) del tipo fill que redimensiona la imagen y elimina los excedentes utilizando los tamaños proporcionados.

Los nombres y los parámetros de las otras manipulaciones o efectos que podemos aplicar sobre nuestras imágenes se pueden consultar en la documentación del sitio de Cloudinary.

3. Consumo de API desde el frontend

Vamos a incorporar los datos que devuelve nuestra API de novedades en nuestro sitio. Para esto es necesario utilizar fetch, una API que ya incluye el navegador. En nuestro caso de uso bastará con pasarle la URL a la cual vamos a llamar. En nuestro caso http://localhost:3000/api/novedades.

Es importante prestar atención a que ahora nuestro componente de página está declarado como asíncrono para poder utilizar await, en este caso, en la llamada de fetch y en la conversión de estos datos a JSON como vemos en el ejemplo.

A continuación, en el archivo **novedades/page.js**

```
import NovedadItem from "@/components/NovedadItem";

export const metadata = {
    title: "Novedades",
    description: "Generated by create next app",
};

export default async function Novedades() {

    const data = await fetch('http://localhost:3000/api/novedades', { cache: 'no-store' });
    const novedades = await data.json();

    return (
        <section className="holder">
            <h2>Novedades</h2>
            {/* <NovedadItem title="titulo de la novedad" subtitle="subtitulo de la novedad"
imagen="https://dummyimage.com/600x400/fff/aaa" body="Lorem ipsum dolor sit amet. " /> */}

            {
                novedades.map(item => <NovedadItem key={item.id}
                    title={item.titulo} subtitle={item.subtitulo}
                    imagen={item.imagen} body={item.cuerpo} />)
            }
        </section>
    )

}
```

Una vez obtenidos los datos podemos recorrerlos con un simple map e ir pasando a cada instancia del componente NovedadItem las propiedades correspondientes a la novedad.

Es muy importante recordar al duplicar elementos en React utilizar la propiedad key, la cual permite a React identificar al elemento de forma rápida.

Conclusión

En resumen, vimos cómo llamar a una fuente de datos externa, en este caso nuestra propia API y cómo utilizar los datos obtenidos para instanciar múltiples copias de nuestro componente, cada una con sus props correspondientes.

En la próxima unidad veremos…

El último paso en todo desarrollo es siempre ponerlo a disposición en algún servidor. Veremos cómo subir las distintas partes de nuestro proyecto tanto a Heroku como a Netlify.



Bibliografía utilizada y sugerida

Artículos de revista en formato electrónico:

Cloudinary. (s.f.). Cloudinary documentation. https://cloudinary.com/documentation/

Handlebars. (s.f.). Handlebars.js: Minimal templating on steroids. https://handlebarsjs.com/

MDN Web Docs. (s.f.). MDN Web Docs. Mozilla. https://developer.mozilla.org/

npm. (s.f.). npmjs.com: Node Package Manager. https://www.npmjs.com/

React. (s.f.). React: Una biblioteca de JavaScript para construir interfaces de usuario. https://es.reactjs.org/

Next.js. (s.f.). Next.js: The React framework. https://nextjs.org/